

3班 202330550601 蒋昱铭

2.

- **死锁发生条件**：当多个进程各自占用部分磁盘空间，且都在等待更多空间，而剩余空间无法满足任何一个进程的需求时，会发生死锁。例如，多个进程占用空间后，每个进程都需更多空间才能完成打印任务，形成循环等待且无足够空间打破僵局。
- **避免方法**：采用资源分配策略，如限制同时打印的进程数，或为每个进程分配固定最大空间，确保不会出现循环等待且资源不足的情况。

9.

存在非法图。例如，资源节点指向自身，这在正常资源分配模型中无意义，因为资源不会自我分配；或进程节点指向非资源节点，违反资源分配模型的结构。

14.

计算每个进程的 $Need = R - C$ ：

- $P1: [1 - 0, 1 - 1, 0 - 1, 2 - 1, 1 - 2] = [1, 0, -1, 1, -1]$ (此处因资源 $RS3$ 、 $RS5$ 出现负数，实际应按算法逻辑检查可行性)。
 - $P2: [0 - 0, 1 - 1, 0 - 0, 2 - 1, 1 - 0] = [0, 0, 0, 1, 1]$ ，可用资源 $A = (0, 1, 0, 2, 1)$ 满足 $P2$ 的 $Need$ ，执行 $P2$ 后 $A = (0, 2, 0, 3, 1)$ 。
 - $P3: [0 - 0, 2 - 0, 0 - 0, 3 - 0, 1 - 1] = [0, 2, 0, 3, 0]$ ，满足 A ，执行 $P3$ 后 $A = (0, 2, 0, 3, 2)$ 。
 - $P4: [0 - 2, 2 - 1, 1 - 0, 1 - 0, 0 - 0] = [-2, 1, 1, 1, 0]$ (负数表示不可行)。
- 最终 $P1$ 、 $P4$ 无法执行，**系统存在死锁，死锁进程为 $P1$ 和 $P4$ 。**

22.

不可能发生死锁。系统有两个进程和三个资源，每个进程最多需两个资源。即使一个进程占用两个资源，另一个进程仍可占用一个资源并执行，完成后释放资源，打破等待链，故不会死锁。

26.

计算 $Need = Max - Allocated$ ：

- $A : [0, 1, 0, 0, 2], B : [0, 2, 1, 0, 0], C : [1, 0, 3, 0, 0], D : [0, 0, 1, 1, 1]$ 。

当 $x = 2$ 时:

- 可用资源 $A = (0, 0, 2, 1, 1)$ 满足 D 的 $Need$, 执行 D 后 $A = (1, 1, 3, 2, 1)$ 。
- C 的 $Need$ 满足, 执行 C 后 $A = (2, 2, 3, 3, 1)$ 。
- A 、 B 依次满足。故 x 的最小值为 **2**。

31.

改进方案: 进程请求新资源前, 先尝试锁定新资源 (如预分配机制), 确保获得新资源后再释放旧资源, 避免旧资源被其他进程抢占。或采用两阶段提交, 先申请新资源, 成功后释放旧资源, 否则保留旧资源。

34.

- **死锁**: 多个进程循环等待对方释放资源, 均无法推进, 处于阻塞状态。
- **活锁**: 进程未阻塞, 但不断重试操作, 因相互干扰始终无法获得资源推进 (如持续交换资源却无进展)。
- **饥饿**: 进程因资源分配策略 (如低优先级) 长期无法获得足够资源执行, 虽未阻塞但无法推进。

补充题

(1) 判断状态是否安全

安全状态的定义是存在一个安全序列, 使每个进程都能顺利执行并释放资源。

- 初始可用资源为 $(1, 6, 2, 2)$ 。
- **P1**: $Need_{P1} = (0, 0, 1, 2)$, 满足 $Available \geq Need_{P1}$ 。执行 P1 后, 释放资源, 可用资源变为 $1 + 0, 6 + 0, 2 + 3, 2 + 2 = (1, 6, 5, 4)$ 。
- **P4**: $Need_{P4} = (0, 6, 5, 2)$, 满足 $Available \geq Need_{P4}$ 。执行 P4 后, 释放资源, 可用资源变为 $1 + 0, 6 + 3, 5 + 3, 4 + 2 = (1, 9, 8, 6)$ 。
- **P5**: $Need_{P5} = (0, 6, 5, 6)$, 满足 $Available \geq Need_{P5}$ 。执行 P5 后, 释放资源, 可用资源变为 $1 + 0, 9 + 0, 8 + 1, 6 + 4 = (1, 9, 9, 10)$ 。
- **P2**: $Need_{P2} = (1, 7, 5, 0)$, 满足 $Available \geq Need_{P2}$ 。执行 P2 后, 释放资源, 可用资源变为 $1 + 1, 9 + 0, 9 + 0, 10 + 0 = (2, 9, 9, 10)$ 。
- **P3**: $Need_{P3} = (2, 3, 5, 6)$, 满足 $Available \geq Need_{P3}$ 。

存在安全序列 $P1 \rightarrow P4 \rightarrow P5 \rightarrow P2 \rightarrow P3$, 因此**状态是安全的**。

(2) 判断 P3 的请求 (1, 2, 2, 2) 是否可批准

- 检查请求是否超过 $Need_{P3}$: $Need_{P3} = (2, 3, 5, 6)$, $(1, 2, 2, 2) \leq Need_{P3}$, 未超过。
- 检查可用资源是否满足请求: $Available = (1, 6, 2, 2)$, $(1, 2, 2, 2) \leq Available$, 暂时满足。
- 假设分配, 可用资源变为 $1 - 1, 6 - 2, 2 - 2, 2 - 2 = (0, 4, 0, 0)$ 。
- 此时检查安全序列:
 - $P1$: $Need_{P1} = (0, 0, 1, 2)$, $C = 0$ 不满足。
 - $P2$: $Need_{P2} = (1, 7, 5, 0)$, $A = 0$ 不满足。
 - $P3$: $Need_{P3} = (1, 1, 3, 4)$, $C = 0$ 不满足。
 - $P4$: $Need_{P4} = (0, 6, 5, 2)$, $C = 0$ 不满足。
 - $P5$: $Need_{P5} = (0, 6, 5, 6)$, $C = 0$ 不满足。

无进程能执行, 系统将死锁。因此, **P3 的请求 (1, 2, 2, 2) 不能批准。**